Advanced Robotics
Homework 3

Yusif Razzaq
GitHub Repo ↗

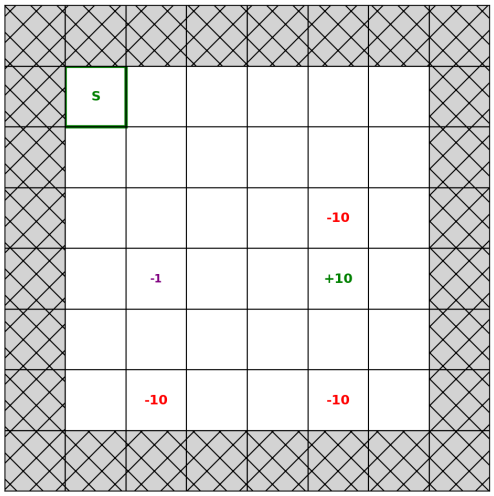
October 2025

1 Introduction

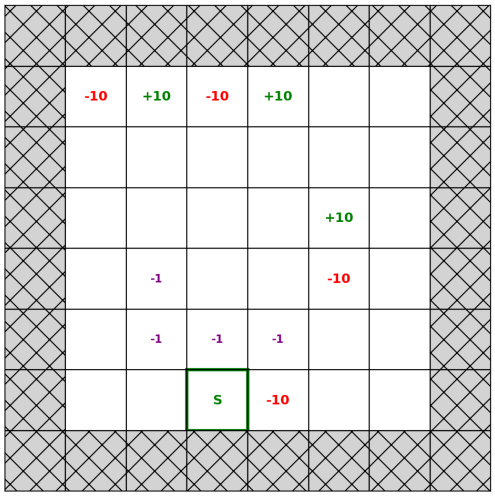
This assignment investigates the use of Markov Decision Processes (MDPs) in two different environments. Given this abstract representation, several methods, namely policy iteration and value iteration, were implemented to arrive at an optimal policy, mapping each state in the environment to an optimal action. The goal is to allow the agent to make optimal decisions with respect to the abstract representation, characterized by transition and reward functions, such that the ensuing runs will maximize expected rewards.

2 Gridworld Environment

In this environment, we consider a simple gridworld where the agent can take steps in any cardinal direction (or stay in place) within the bounds of the state space (an 8x8 grid). A few reward states (both positive and negative) were distributed around the grid as shown in Figure 1.



(a) `Gridworld-v0` Environment



(b) `Gridworld-v1` Environment

Figure 1: Initial environmental setup with terminal states in red/green, and non-zero reward states in purple.

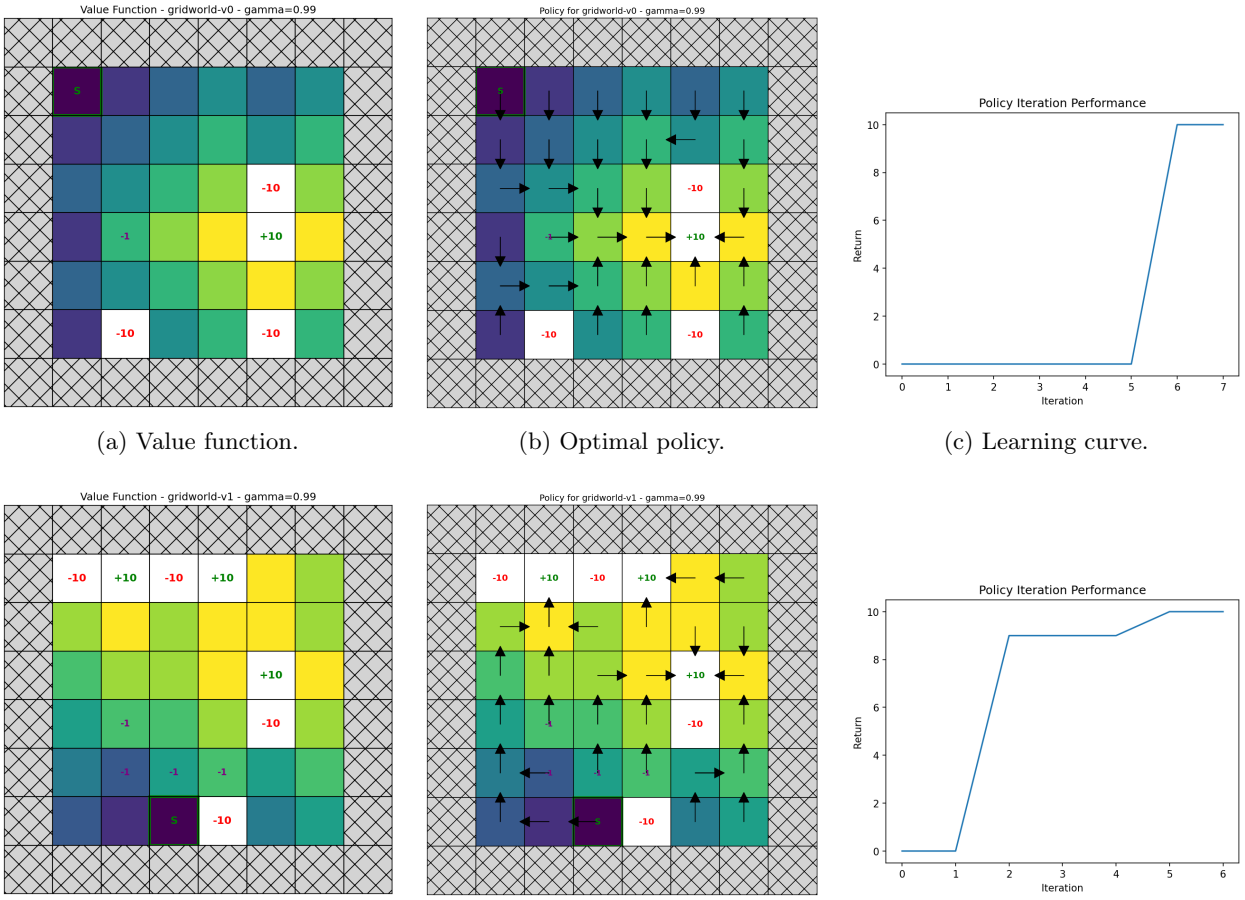
2.1 Policy Iteration

Given this problem setup, the first approach to finding an optimal policy was to implement Policy Iteration (PI). The key steps to this method include first initializing a random policy and value function. Then, for

each iteration, the value of every state is determined using the current policy to evaluate the value function under that policy.

Next, a policy improvement stage is performed, such that, given the new value function, the new policy chooses the action at each state that maximizes the expected reward. This process of policy evaluation and iteration continues until no further changes are made to the policy (i.e., convergence).

The results from this approach, given a discount factor of $\gamma = 0.99$, are shown in Figure 2 for the first (top row) and second (bottom row) environments. The resulting value function is shown in a blue-yellow color scale, where yellow depicts a higher reward at the state. The optimal policy is shown as arrows directing the agent to take a step into an adjacent cell, leading to higher value states and, eventually, the +10 reward.



(a) Value function. (b) Optimal policy. (c) Learning curve.

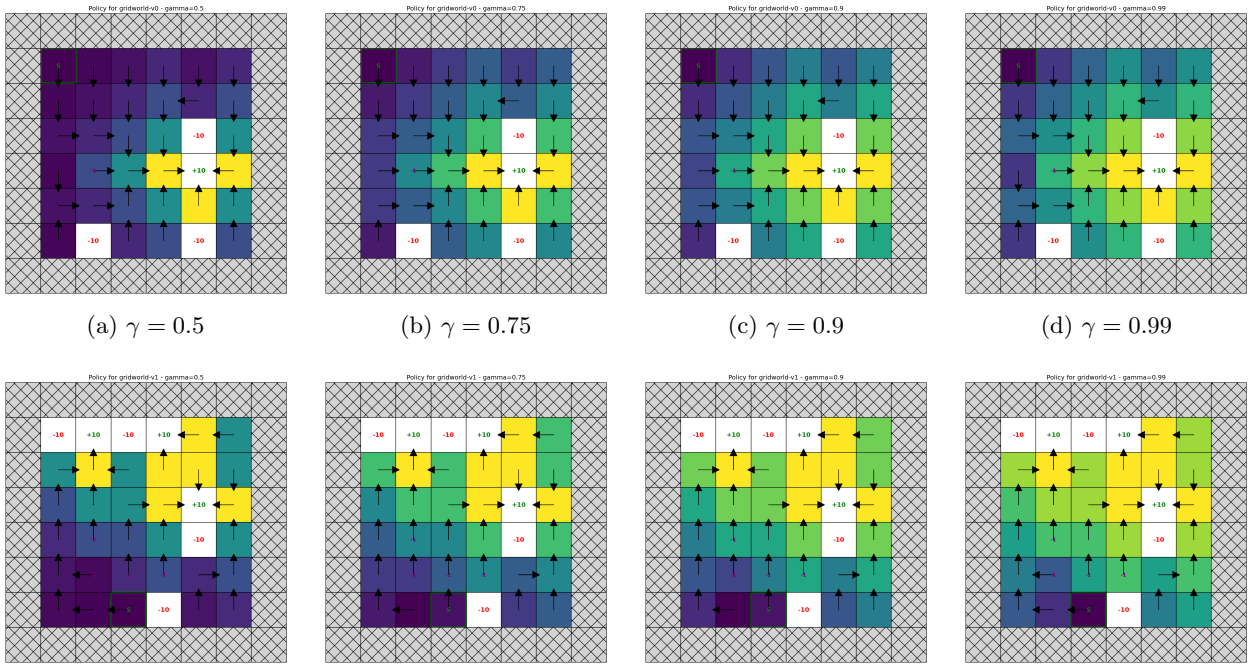
(d) Value function. (e) Optimal policy. (f) Learning curve.

Figure 2: Results for environments `GW-v0` (top row) and `GW-v1` (bottom row) under $\gamma = 0.99$. Each row shows the value function, optimal policy, and learning curve demonstrating convergence.

Finally, a learning curve is shown, depicting the return when following the policy at every step towards convergence. The trend increases monotonically as the policy is refined according to a value function closer to the true optimal. As expected, both gridworlds exhibited similar solutions given their respective definitions, showing that policy iteration is capable of solving any such environment.

2.1.1 Effects of Discount Factor γ

Next, the effects of altering the discount factor γ were considered. In order to test the resulting policy, γ was incremented using the following parameters: $\gamma = [0.5, 0.75, 0.9, 0.99]$. The resulting policies, overlaid onto their respective value functions, are presented in Figure 3 for `GW-v0` (top) and `GW-v1` (bottom).



(a) $\gamma = 0.5$ (b) $\gamma = 0.75$ (c) $\gamma = 0.9$ (d) $\gamma = 0.99$

(e) $\gamma = 0.5$ (f) $\gamma = 0.75$ (g) $\gamma = 0.9$ (h) $\gamma = 0.99$

Figure 3: Comparison of resulting value functions and policies under varying discount factors $\gamma \in \{0.5, 0.75, 0.9, 0.99\}$ for environments `GW-v0` (top row) and `GW-v1` (bottom row). Higher γ values yield longer-horizon policies that prioritize future rewards.

The results show that changing γ does impact the resulting optimal policy. This is due to the calculation of value being dependent on the value of successor states, scaled by γ. Therefore, for lower values of γ, the policy is more likely to go through states with -1 reward since the value of the +10 states doesn't "travel" as far, making a longer detour appear preferable. These effects are more pronounced in `GW-v1`.

2.2 Value Iteration

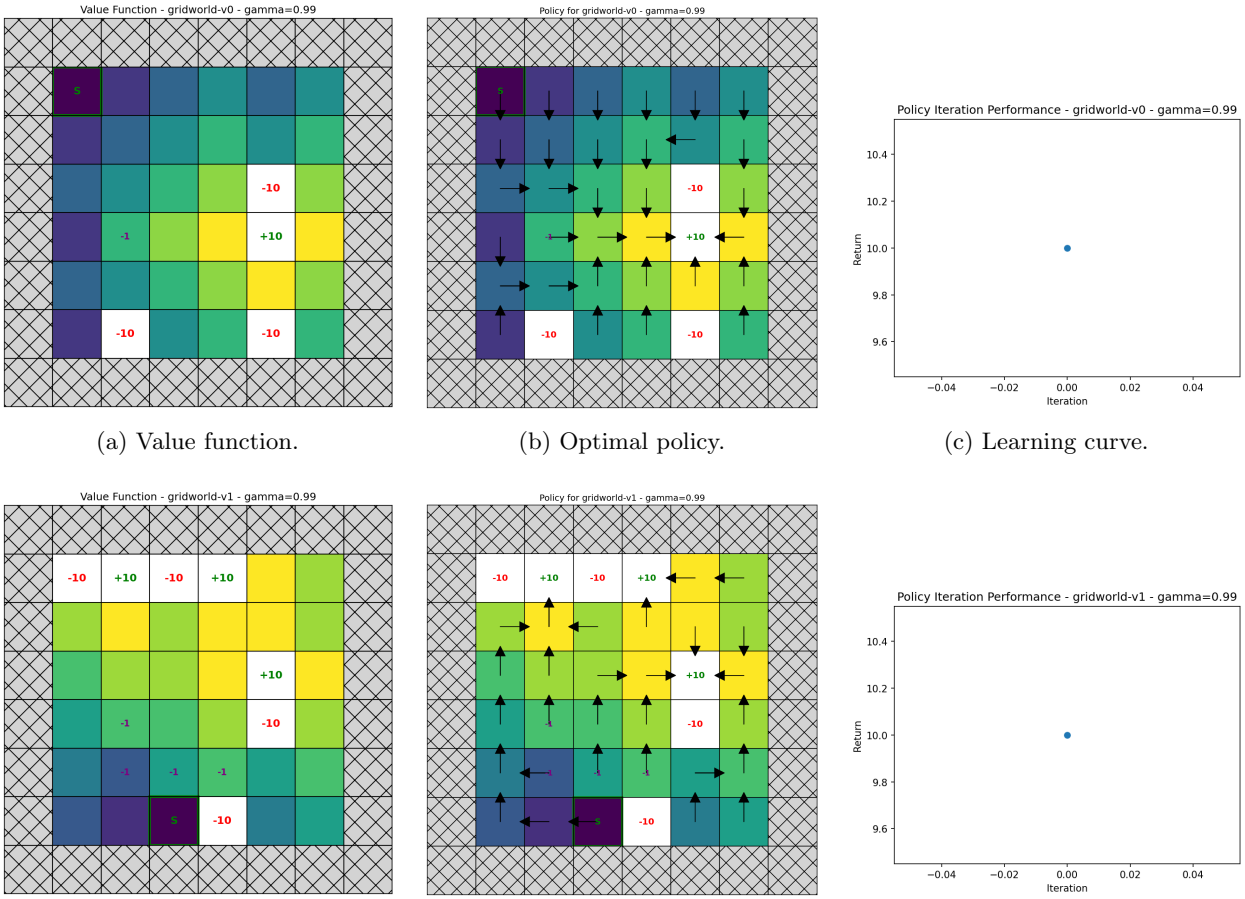
The same gridworld environments were then solved using value iteration (VI). This method bypasses the need to compute intermediate policies by first finding the optimal value function, performed by iteratively applying Bellman's operator until convergence. Once the value function is found, a single sweep is done to find the action at every state that maximizes value, hence returning an optimal policy with respect to that value function.

The results are presented in Figure 4. These computed value functions and their accompanying policies are identical to the results found using Policy Iteration. This is expected, as both methods are guaranteed to converge onto the optimal policy at the limit.

2.2.1 Performance Comparison: PI vs. VI

The two solvers were compared according to the time it took for a solution to be returned. The results are summarized in Table 1, which shows the average computation time over the various γ values tested. This shows that VI was considerably faster (up to an order of magnitude), particularly for high values of γ, which took PI a long time to solve.

This speed-up can be attributed to the fact that VI does not have to solve for the value function every time the policy is updated; rather, it solves directly for the optimal value function and then determines the optimal policy immediately after.



(a) Value function. (b) Optimal policy. (c) Learning curve.

(d) Value function. (e) Optimal policy. (f) Learning curve.

Figure 4: Results of the Value Iteration method for environments `GW-v0` (top row) and `GW-v1` (bottom row). The computed value functions, optimal policies, and learning progressions match those obtained using Policy Iteration.

Method	Map	Average Time (ms)	Time Range (ms)
Policy Iteration (PI)	`GW-v0`	101	31 – 251
Policy Iteration (PI)	`GW-v1`	99	31 – 242
Value Iteration (VI)	`GW-v0`	17	17 – 18
Value Iteration (VI)	`GW-v1`	17	17 – 17

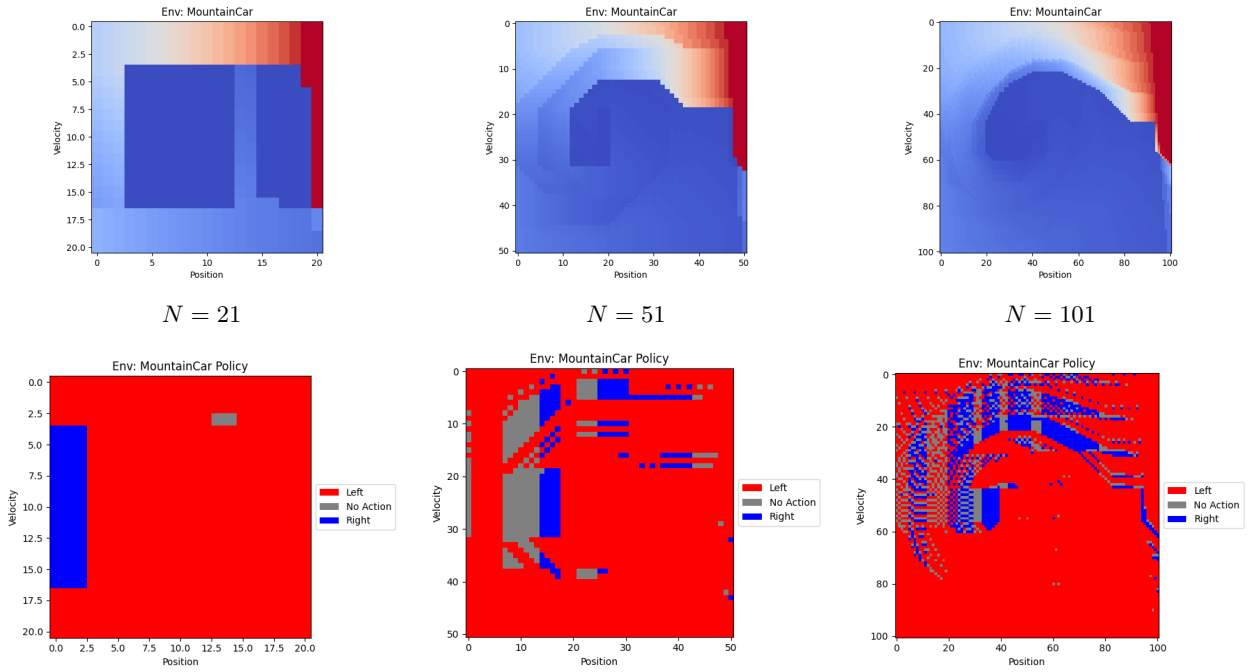
Table 1: Comparison of Policy Iteration (PI) and Value Iteration (VI) computation times for two gridworld maps (in milliseconds).

3 Mountain Car Environment

For this environment, a more realistic scenario was considered. The setup describes a car in a 2-D continuous state space ($position = p, velocity = v$), with the goal of reaching the top of the mountain by accelerating left, right, or not accelerating.

However, since MDPs must be defined over discrete and finite states for standard solvers to work, the state space was first discretized. Once discretized, a mapping between continuous states and the discrete states representing them was established to enable complete transition logic. Two methods were tested: nearest-neighbor and linear interpolation.

3.1 Nearest-Neighbor Interpolation



$N = 21$ $N = 51$ $N = 101$

$N = 21$ $N = 51$ $N = 101$

Figure 5: Results for the nearest-neighbor approximation under different discretization resolutions ($N = \{21, 51, 101\}$). The top row shows the state-value heatmaps, while the bottom row displays the corresponding optimal policies obtained via Value Iteration.

This method simply maps each continuous state to the closest discrete state, where distance is measured with respect to the center of the discrete cell. Thus, a deterministic transition system is defined, in which each continuous state deterministically transitions to the discrete cell with the nearest centroid.

The method was tested with discretization resolutions along each dimension: $N = \{21, 51, 101\}$. Given

the transition logic, Value Iteration was implemented to solve for the optimal value function, followed by the corresponding policy. The resulting state-value heatmaps are shown in the top row of Figure 5, and the policies are shown in the bottom row.

This method struggled to solve the problem at low resolutions, as the reward for the optimal policy was consistently close to -200 (the maximum number of steps, meaning the goal was not reached). However, when 101 bins were used, the problem was reliably solved, with rewards around -191.

A limitation of this approach is the state-space explosion with finer discretization. Although policy performance improved, this came at a substantial computational cost. To solve the MDP in reasonable time, vectorization using Python's NumPy package was heavily utilized, replacing slow loops with optimized matrix operations.

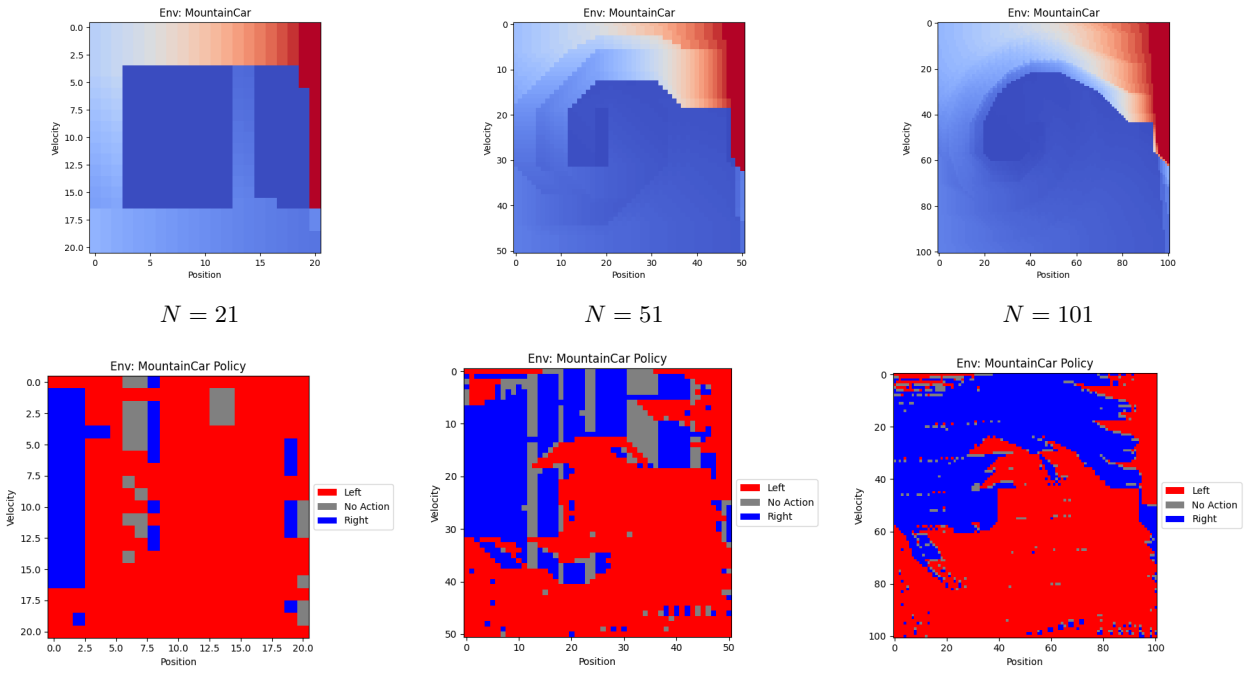
3.2 Linear Interpolation

Next, the same problem was solved using a linear interpolation method for transition dynamics. To determine the successor state of the agent, this method first considers all neighboring grid cells relative to the continuous state. Then, based on the distance of each cell's centroid to the continuous state, a weight is applied as follows:

$$w_i^- = \frac{1}{|\vec{x} - \vec{c}_i|}$$

where $\vec{x}$ is the continuous state and $\vec{c}_i$ is the centroid of a neighboring grid cell. The weights are then normalized so that their sum equals one.

The results of this interpolation method are presented in Figure 6. The policies obtained using linear interpolation showed notable improvement over the nearest-neighbor approach, achieving rewards as high as -109 with $N = 101$ bins.



$N = 21$ $N = 51$ $N = 101$

$N = 21$ $N = 51$ $N = 101$

Figure 6: Results for the linear interpolation approach under different discretization resolutions ($N = \{21, 51, 101\}$). The top row shows the state-value heatmaps, while the bottom row displays the corresponding optimal policies obtained via Value Iteration. This method yielded smoother policies and improved performance compared to the nearest-neighbor approach, achieving rewards as high as -109 for $N = 101$.

Table 2 compares the performance of the two interpolation methods across various resolutions. Linear interpolation outperforms nearest-neighbor interpolation without substantially increasing computation time.

Bin Size	**Algorithm**	**Performance**	**Time (s)**
21	nn	-200	0.27
21	linear	-200	0.62
51	nn	-200	6.87
51	linear	-200	7.27
101	nn	-191	42.94
101	linear	-109	44.18

Table 2: Comparative performance in the Mountain Car environment for different bin sizes and interpolation algorithms.

4 Conclusion

In this assignment, we implemented various MDP solvers across several environments and abstraction methods. First, Policy Iteration and Value Iteration were compared in a Gridworld environment, and both were able to arrive at the same optimal policy. Value Iteration was considerably faster. We also demonstrated that the discount factor γ affects both the resulting optimal policy and the convergence time, as higher γ values effectively increase the value propagation across states.

Next, the Mountain Car environment, which is natively defined in continuous space, was considered. Various discretization resolutions were tested to determine their effects. Higher-resolution discretizations better approximate the true continuous dynamics, although computation time scales exponentially. Finally, two methods for representing the transition dynamics were compared. Results showed that linear interpolation, which defines a distribution over possible state transitions, outperforms nearest-neighbor mapping.[1]

[1] AI tools were used to format the plots and tables, and to correct grammatical mistakes in the report.